UNIVERSIDAD
DE GRANADA

Universidad de Granada

Escuela Técnica Superior de Ingeniería Informática y de Telecomunicación

Práctica 3: Representación de dominios y resolución de problemas con técnicas de planificación

Alumno: Joaquín Cruz Lorenzo
Grado en Ingeniería Informática
Curso 2024–2025

Ejercicio 1: Modelo base (viajar y extraer recursos)

El objetivo de este primer ejercicio fue establecer un dominio PDDL fundamental, denominado "modelo base", que representara un mundo simplificado con personajes, localizaciones, recursos y las acciones básicas de movimiento y extracción, cumpliendo con los requisitos mínimos del enunciado.

0.1. Diseño del Dominio (`dominio1.pddl`)

El dominio se diseñó con una estructura de tipos y predicados clara, y dos acciones esenciales:

0.1.1. Tipos

Se definieron los tipos principales `Personaje` (con subtipos `Enano` y `Hobbit`), `Recurso` (con subtipos como `Madera`) y `Localizacion`. Esta jerarquía permite una modelización precisa y la restricción de acciones a tipos específicos de personajes.

0.1.2. Predicados Esenciales

- `(en ?x - (either Personaje Recurso) ?l - Localizacion)`: Para ubicar tanto personajes como nodos de recursos específicos.
- `(camino ?l1 - Localizacion ?l2 - Localizacion)`: Define la conectividad del mapa.
- `(trabajando ?p - Personaje ?l - Localizacion ?r - Recurso)`: Indica el estado objetivo donde un personaje está extrayendo un recurso.
- `(disponible ?p - Personaje)`: Controla la disponibilidad de un personaje para actuar.

0.1.3. Acciones

Conforme al enunciado, se implementaron únicamente dos acciones:

- `Viajar`: Permite a un `Personaje` disponible moverse entre dos `Localizacione`s conectadas, actualizando su ubicación.
- `ExtraerRecurso`:
  - El parámetro del personaje está tipado como `Enano` (`?p - Enano`), implementando la restricción de que "sólo los Enanos pueden extraer".
  - Requiere que el enano y el recurso estén en la misma `Localizacion` y el enano esté `disponible`.
  - Como efecto, el enano queda `(not disponible)` y pasa a estar `(trabajando)` con el recurso específico.

0.2. Diseño del Problema (`problema1.pddl`)

El fichero de problema instancia el dominio con la configuración solicitada:

0.2.1. Objetos

Se definieron los personajes `Enano1`, `Enano2`, y `Hobbit1`; todas las localizaciones del mapa; y el recurso `MaderaFangorn` de tipo `Madera`.

0.2.2. Estado Inicial (':init')

- Se ubicó a `Enano1` en `Tharbad` (disponible), `Enano2` en `Isengard` (no disponible, "ocupado con una misión diplomática"), y `Hobbit1` en `Isengard` (disponible).
- El recurso se situó con `(en MaderaFangorn Fangorn)`.
- Se establecieron todos los caminos bidireccionales del mapa.

0.2.3. Objetivo (':goal')

El objetivo se definió como `(trabajando Enano1 Fangorn MaderaFangorn)`, buscando que `Enano1` extraiga el nodo de madera específico en su localización, cumpliendo así con la meta de "generar recursos de tipo Madera".

0.2.4. Decisiones Clave de Diseño

El diseño se adhirió a los requisitos del "modelo base", enfocándose en la representación de recursos como objetos individuales con ubicación y la restricción de la extracción a los `Enano`s. Esto sienta una base sólida para futuras extensiones.

Ejercicio 2: Formar Comunidad reducida y destruir el Anillo Único

En este ejercicio se planteó una extensión del modelo anterior con el objetivo de formar una Comunidad del Anillo reducida y llevar el Anillo Único al Monte del Destino para destruirlo. Para ello, se incorporaron nuevas entidades, condiciones y acciones al dominio, así como un nuevo objetivo simbólico: `(anilloDestruido)`.

Modelado y decisiones de diseño

Tipos y predicados añadidos. Se introdujeron nuevos tipos de personaje (Mago y Elfo), así como nuevos predicados que permiten modelar las condiciones de pertenencia a la Comunidad (`comunidadFormada`, `miembroComunidad`), localización del anillo, y pertenencia de objetos a un personaje (`lleva`).

También se añadió el predicado `lugarDestruccion` para indicar que Orodruin es el único sitio válido donde el anillo puede ser destruido, cumpliendo el requerimiento del enunciado.

Nuevas acciones. Se implementaron las acciones requeridas:

- **formarComunidad**: Permite constituir la Comunidad a partir de dos personajes en la misma localización, cumpliendo con el diseño propuesto (Hobbit y Mago).
- **viajarComunidad**: Reemplaza la acción individual de viaje una vez formada la Comunidad. Asegura que todos los miembros se desplazan en conjunto.
- **recogerObjeto**: Permite a un personaje recoger un recurso en una localización, modelando el proceso de recolecta del Anillo, Espada y Mithril.
- **destruirAnillo**: Acción final que solo puede ejecutarse si el personaje lleva los tres objetos (Anillo, Espada y Mithril), está en el lugar de destrucción, y es Hobbit miembro de la Comunidad.

Consideraciones clave

- La Comunidad solo puede formarse si ambos personajes están en la misma localización y son de tipos compatibles (Hobbit + Mago).
- Una vez formada, todos los viajes deben hacerse en conjunto, con lo cual la acción original `Viajar` se inhabilita para miembros de la Comunidad.
- El Anillo, la Espada y el Mithril fueron definidos como objetos de tipo Recurso, lo que simplifica su recogida con la acción `recogerObjeto`.
- Se validó que el portador del anillo fuese un Hobbit y miembro de la Comunidad, según las restricciones del enunciado.
- El plan resultante optimiza el número de acciones requeridas para formar la comunidad, recoger los objetos, y destruir el anillo en Orodruin.

Resultado

La solución implementada cumple con todas las restricciones establecidas en el enunciado, incluyendo las condiciones necesarias para recoger y destruir el anillo. El modelo se mantiene modular, reutilizando predicados del ejercicio anterior y extendiendo el dominio de manera escalable.

Ejercicio 3: Ampliación de la Comunidad y análisis de rendimiento

En este ejercicio se parte del planteamiento del Ejercicio 2, con el objetivo de analizar cómo afecta el tamaño de la Comunidad a los tiempos de planificación y a la longitud de los planes generados por el planner.

Diseño del experimento

Para ello se han definido cuatro problemas distintos, en los que se incrementa progresivamente el número de integrantes de la Comunidad del Anillo. Las dimensiones probadas han sido las siguientes:

- 1 hobbit y 1 mago
- 2 hobbits y 1 mago
- 3 hobbits y 1 mago
- 3 hobbits, 1 mago y 1 enano

En cada uno de estos casos se ha adaptado el predicado de formación de comunidad y los parámetros de la acción `viajarComunidad` para reflejar correctamente a todos los integrantes involucrados. Además, se ha mantenido la lógica de destrucción del anillo, exigiendo que se hayan recogido los objetos (anillo, mithril y espada) por parte del mismo portador (un hobbit miembro de la comunidad), y que dicho portador llegue a la localización de destrucción en Orodruin.

Resultados obtenidos

La siguiente tabla resume los resultados obtenidos en cuanto a tiempo de planificación (*Planner Time*) y longitud del plan generado (*Plan Length*):

Tamaño de la Comunidad	**Tiempo de planificación (s)**	**Longitud del plan**
1 hobbit y 1 mago	0.19	12
2 hobbits y 1 mago	2.12	14
3 hobbits y 1 mago	16.57	15
3 hobbits, 1 mago y 1 enano	342.36	18

Análisis y discusión

Los resultados muestran una clara tendencia de crecimiento exponencial en el tiempo de planificación a medida que se incrementa el número de personajes en la comunidad. Esto es coherente con lo esperado en planificación automática, donde el espacio de búsqueda se ve afectado por la combinatoria derivada de:

- Aumento del número de objetos (más personajes, más posibles combinaciones de ubicaciones).
- Mayor número de parámetros en las acciones (por ejemplo, la acción `formarComunidad` y `viajarComunidad` aumenta su aridad).
- Reutilización de predicados que deben comprobar condiciones de pertenencia o sincronización entre todos los miembros.

Aunque la longitud del plan sólo se incrementa moderadamente (de 12 a 18 pasos), el tiempo necesario para encontrar dicho plan crece drásticamente. El último caso (3 hobbits, 1 mago y 1 enano) supera los 5 minutos de planificación, incluso manteniendo la misma estructura de acciones que los casos anteriores. Esto refleja la naturaleza combinatoria del problema y cómo el crecimiento en aridad y cardinalidad impacta directamente en el rendimiento del planner.

Predicción hipotética

Si incorporáramos una comunidad formada por 3 hobbits, 1 mago, 1 elfo, 1 enano y 2 humanos, es decir, 8 miembros, el planner tendría que evaluar más de 8! permutaciones posibles solo para tareas secuenciales. Teniendo en cuenta la tendencia observada, estimamos que el tiempo de planificación podría superar los 30 minutos

Conclusión

Este ejercicio demuestra claramente cómo el tamaño de la comunidad impacta en la escalabilidad del dominio. En planificación automática, más objetos y más parámetros aumentan el espacio de búsqueda exponencialmente. En consecuencia, resulta esencial diseñar dominios lo más compactos posible y considerar estrategias de partición o preprocesamiento si se desea escalar a escenarios más complejos.

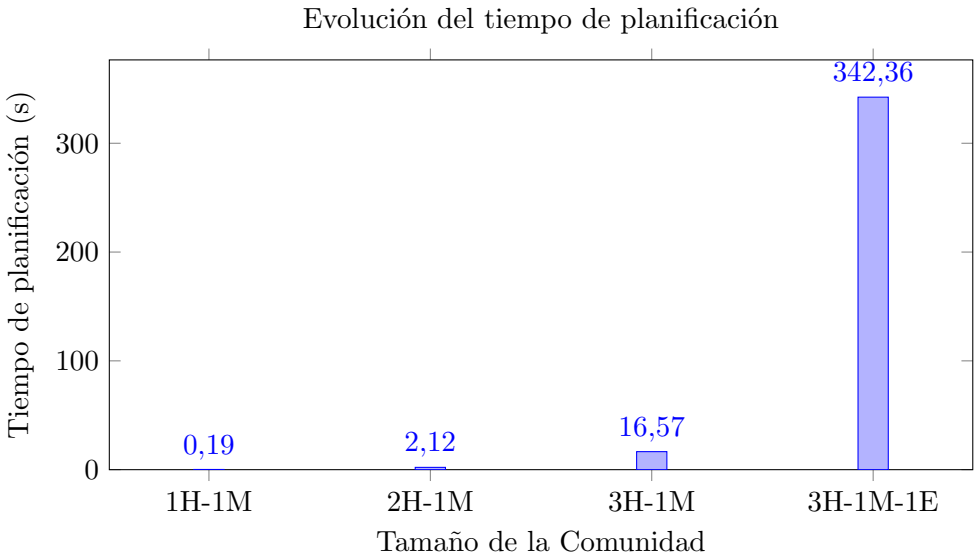

Figura 1: Tiempo de planificación según tamaño de la Comunidad

Ejercicio 4: Construcción de Fortaleza a partir de múltiples recursos

El dominio se centra en modelar las capacidades diferenciadas de los personajes y las dependencias en la producción de recursos.

Tipos y Predicados Clave: Se definieron tipos jerárquicos para `Personaje` (con subtipos `Hobbit`, `Enano`, `Humano`, `Mago`), `Recurso`, `Edificio` (`Extractor`, `Fortaleza`) y `Localizacion`. Los predicados esenciales incluyen `(en)`, `(disponible)`, `(camino)`, `(trabajando ?p ?loc ?r)`, `(recurso-disponible ?loc ?r)`, y `(requiere-recurso ?e ?r)`, además de los predicados de tipo `(es-...)`.

Acciones y Decisiones de Diseño:

- `ExtraerRecurso`: Implementa la lógica condicional donde los `Hobbits` extraen `Alimento`; para otros recursos (`Mineral`, `Madera`, `Mithril`), se requiere que todos los Hobbits estén previamente trabajando (verificado con `forall-exists`). La extracción de `Mithril` por un `Enano` necesita un `Extractor` construido localmente. El personaje queda `(trabajando)`.

- `Construir`: Un `Mago` construye `Fortaleza` y un `Humano` un `Extractor`. El constructor debe estar disponible y en el lugar.
  - *Decisión Clave (Regla de Recursos Global):* Para esta versión (que produce el plan de 24 pasos), se adoptó que los recursos necesarios solo deben estar siendo trabajados por algún personaje en *cualquier localización*, no necesariamente en el sitio de construcción. Esto se modela con:

    ```
    (forall (?r_nec) (imply (requiere-recurso ?e ?r_nec)
        (exists (?p_trab ?l_trab) (trabajando ?p_trab ?l_trab ?r_nec))))
    ```

  - El constructor recupera su disponibilidad tras la acción. Se recomienda añadir `(not (edificio-constru ?e ?loc))` como precondición.

0.2.5. Diseño del Problema (`problema4.pddl`)

La configuración del problema fue crucial para obtener el plan de 24 pasos, forzando una estrategia específica.

Objetos y Estado Inicial:

- Se definieron 4 Hobbits, 2 Magos, 1 Elfo, 2 Enanos, 2 Humanos, y los edificios `Fortaleza1` y `Extractor1`, junto con todas las localizaciones del mapa detallado proporcionado por el compañero.
- *Distribución de Recursos y Conectividad Estratégica:*
  - `Mineral`: Disponible solo en `Moria` y `Erebor` (no en `Edoras`, donde inician los Humanos).
  - `Mithril`: Disponible solo en `Moria`.
  - `Madera`: En `Fangorn`, `Lothlorien`, `Mirkwood`. `Alimento` en `Hobbiton` y `Lothlorien`.
  - El mapa utilizado carece de un camino directo `Erebor <->Moria`, obligando a `Enano2` a una ruta más larga. Similarmente, las rutas de los Humanos desde `Edoras` a `Moria` son extensas.
  - No hay recursos iniciales (`Madera`, `Mineral`, `Mithril`) en `Isengard`.
- Todos los personajes inician disponibles.

Objetivo (‘:goal‘): `(edificio-construido Fortaleza1 Isengard)`.

0.2.6. Solución y Estrategia del Plan (24 pasos)

Con esta configuración, se obtiene un plan de 24 pasos. La estrategia general implica: 1. Hobbits activan Alimento. 2. `Enano1` extrae Madera en Fangorn. 3. Humanos viajan a Moria, uno extrae Mineral y el otro construye `Extractor1`. 4. `Enano2` viaja a Moria (ruta larga) y extrae Mithril. 5. `Mago2` en Isengard construye `Fortaleza1` usando los recursos globalmente trabajados. La combinación de la regla de construcción global y las restricciones del problema (ubicación de recursos y mapa) guían al planificador hacia esta solución específica.

Ejercicio 5: respuesta a la pregunta

Para implementar un método que garantice, o al menos se aproxime a, que el plan devuelto sea óptimo en número de pasos, es necesario modificar la estrategia de búsqueda empleada por el planificador. En particular, se pueden considerar las siguientes opciones:

- **Utilizar una métrica de coste basada en la cantidad de acciones ejecutadas.** En PDDL esto se hace especificando `(:metric minimize (total-cost))` y asegurándose de que todas las acciones tengan un coste uniforme (por ejemplo, 1 unidad). Así, el planificador intentará minimizar el número total de acciones.

- **Escoger un alias de planificador que priorice planes más cortos.** En Fast Downward, se puede utilizar `--alias seq-opt-lmcut`, que está diseñado para encontrar planes óptimos en número de pasos. Este alias aplica búsqueda A* con la heurística LM-cut, garantizando la optimalidad del plan.
- **Evitar el uso de estrategias heurísticas no óptimas.** Por ejemplo, `seq-sat-lama-2011` está optimizado para velocidad y no garantiza optimalidad. Si se desea un plan óptimo, debe evitarse este tipo de configuración, ya que podría devolver un plan válido pero subóptimo.
- **Limitar las acciones redundantes.** Otra medida complementaria es evitar en el modelo que un personaje viaje innecesariamente o que se pueda volver a asignar a un recurso si ya estaba trabajando. Esto reduce la ramificación del espacio de búsqueda.

En resumen, la combinación de una métrica adecuada, una heurística óptima (como lm-cut), y un dominio bien definido sin redundancias, permite al planificador generar soluciones más próximas a la optimalidad o incluso óptimas, según el alias seleccionado.